

Name: Kshitiz Reddy.M

Reg No: A26MTAI0008

Lab 5: Designing and Training Neural Networks — Logic Gates, Backpropagation and Vanishing Gradient

Dataset/task: XOR, AND, OR logic gates (4 samples each) for Q1-Q3, and `make_moons` for Q4. Everything is implemented from scratch with NumPy — no TensorFlow/Keras/PyTorch, as instructed.

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

np.random.seed(42)
plt.rcParams['figure.figsize'] = (6, 4)
```

Question 1: Logic Gates and Linear Separability

Objective: understand why a hidden layer is needed for XOR.

In [2]:

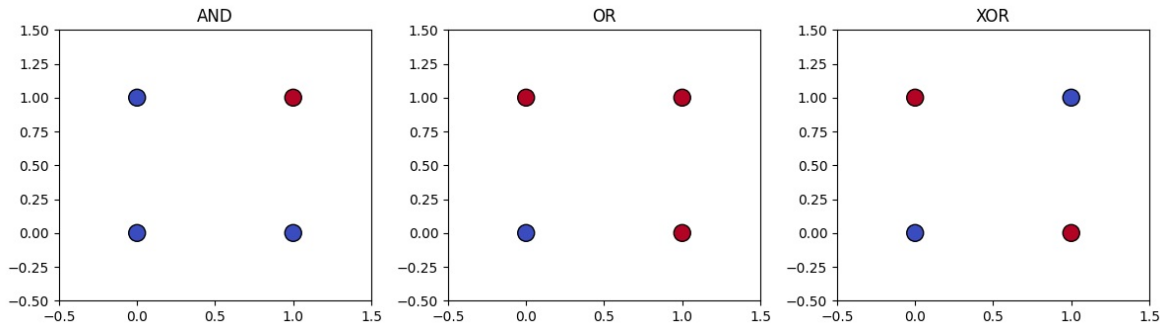
```
# 1. AND, OR, XOR datasets
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
y_or = np.array([0,1,1,1])
y_xor = np.array([0,1,1,0])

print(X)
print("AND:", y_and)
print("OR :", y_or)
print("XOR:", y_xor)

[[0 0]
 [0 1]
 [1 0]
 [1 1]]
AND: [0 0 0 1]
OR : [0 1 1 1]
XOR: [0 1 1 0]
```

In [3]:

```
# 2. Plot the three datasets
fig, axes = plt.subplots(1, 3, figsize=(12, 3.5))
for ax, y, title in zip(axes, [y_and, y_or, y_xor], ["AND", "OR", "XOR"]):
    ax.scatter(X[:,0], X[:,1], c=y, cmap='coolwarm', s=150, edgecolor='k')
    ax.set_title(title)
    ax.set_xlim(-0.5, 1.5)
    ax.set_ylim(-0.5, 1.5)
plt.tight_layout()
plt.show()
```



Which are linearly separable? AND and OR both are -- you can draw one straight line that puts the single "1" class on one side (AND) or the single "0" class on one side (OR). XOR isn't: the two 1s are at (0,1) and (1,0), the two 0s are at (0,0) and (1,1), and those are on opposite corners of the square, so no single straight line separates them.

Why can't a single-layer perceptron learn XOR? A perceptron just computes a weighted sum of the inputs and thresholds it, which is exactly a straight-line (linear) decision boundary. Since XOR needs a non-linear boundary, a single layer can never represent it no matter how the weights are set. You need at least one hidden layer so the network can first bend the input space into a form where the classes become linearly separable, and only then draw a line.

Network design (Q1): input(2) → hidden1(2, +bias) → hidden2(2, +bias) → output(2, +bias), sigmoid activations everywhere. Weight matrices: W1, W2, W3 all 2x2, and each layer gets its own bias row vector.

Question 2: Forward Propagation

In [4]:

```
# 3. Initialize weights and biases (W1,W2,W3 = 2x2, b1,b2,b3 = 1x2)
rng = np.random.RandomState(42)
W1 = rng.randn(2,2) * np.sqrt(1/2); b1 = np.zeros((1,2))
W2 = rng.randn(2,2) * np.sqrt(1/2); b2 = np.zeros((1,2))
W3 = rng.randn(2,2) * np.sqrt(1/2); b3 = np.zeros((1,2))

print("W1", W1.shape, "b1", b1.shape)
print("W2", W2.shape, "b2", b2.shape)
print("W3", W3.shape, "b3", b3.shape)
```

```
W1 (2, 2) b1 (1, 2)
W2 (2, 2) b2 (1, 2)
W3 (2, 2) b3 (1, 2)
```

In [5]:

```
# 4. Sigmoid and its derivative
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)
```

In [6]:

```
# 5. Forward propagation
Z1 = X @ W1 + b1
A1 = sigmoid(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)
Z3 = A2 @ W3 + b3
Yhat = sigmoid(Z3)

# 6. Dimensions of every intermediate variable
print("X :", X.shape)
print("Z1:", Z1.shape, " A1:", A1.shape)
print("Z2:", Z2.shape, " A2:", A2.shape)
print("Z3:", Z3.shape, " Yhat:", Yhat.shape)
```

```
X : (4, 2)
Z1: (4, 2)  A1: (4, 2)
Z2: (4, 2)  A2: (4, 2)
Z3: (4, 2)  Yhat: (4, 2)
```

In [7]:

```
# 7. Walk through one sample (sample 0 = [0,0])
print("Input:", X[0])
print("Z1:", Z1[0], " A1:", A1[0])
print("Z2:", Z2[0], " A2:", A2[0])
print("Z3:", Z3[0], " Output:", Yhat[0])
```

```
Input: [0 0]
Z1: [0. 0.]  A1: [0.5 0.5]
Z2: [0.47555033 0.18854924]  A2: [0.6166966 0.54699816]
Z3: [-0.38396738 0.05645655]  Output: [0.40517037 0.51411039]
```

In [8]:

```
# 8. MSE loss, one-hot encode XOR target (class0 -> [1,0], class1 -> [0,1])
Y = np.array([[1,0],[0,1],[0,1],[1,0]], dtype=float)

loss = np.mean((Y - Yhat) ** 2)
print("Initial loss (untrained network):", loss)
```

```
Initial loss (untrained network): 0.2549510135394929
```

Loss starts around 0.25 -- makes sense, the network is untrained and all outputs are close to $\text{sigmoid}(0) = 0.5$, so squared error against a 0/1 target averages out to roughly $(0.5)^2 = 0.25$ before any learning happens.

Question 3: Backpropagation and Training on XOR

In [9]:

```
# 9. Backprop -- gradients via chain rule, output -> hidden2 -> hidden1
def backward(X, Y, Z1, A1, Z2, A2, Z3, Yhat, W2, W3):
    N = X.shape[0]
    dA3 = -2/(N*Y.shape[1]) * (Y - Yhat)
    dZ3 = dA3 * sigmoid_derivative(Z3)
    dW3 = A2.T @ dZ3
    db3 = np.sum(dZ3, axis=0, keepdims=True)

    dA2 = dZ3 @ W3.T
    dZ2 = dA2 * sigmoid_derivative(Z2)
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * sigmoid_derivative(Z1)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    return dW1, db1, dW2, db2, dW3, db3
```

In [10]:

```
# quick check that the gradients above are correct, comparing to a numerical estimate
eps = 1e-5
W1c = W1.copy()
Z1c, A1c = X @ W1c + b1, sigmoid(X @ W1c + b1)
Z2c, A2c = A1c @ W2 + b2, sigmoid(A1c @ W2 + b2)
Z3c, Yhatc = A2c @ W3 + b3, sigmoid(A2c @ W3 + b3)
dW1c, *_ = backward(X, Y, Z1c, A1c, Z2c, A2c, Z3c, Yhatc, W2, W3)

i, j = 0, 0
W1_plus = W1c.copy(); W1_plus[i,j] += eps
Z1p, A1p = X @ W1_plus + b1, sigmoid(X @ W1_plus + b1)
Yhatp = sigmoid(sigmoid(A1p @ W2 + b2) @ W3 + b3)
loss_plus = np.mean((Y - Yhatp) ** 2)

W1_minus = W1c.copy(); W1_minus[i,j] -= eps
Z1m, A1m = X @ W1_minus + b1, sigmoid(X @ W1_minus + b1)
Yhatm = sigmoid(sigmoid(A1m @ W2 + b2) @ W3 + b3)
loss_minus = np.mean((Y - Yhatm) ** 2)

numeric_grad = (loss_plus - loss_minus) / (2*eps)
print("Analytic dW1[0,0]:", dW1c[i,j], " Numeric:", numeric_grad)
```

Analytic dW1[0,0]: -3.086312109527005e-05 Numeric: -3.0863120392687904e-05

In [11]:

```
# 10. Training loop -- plain gradient descent, no optimizer
np.random.seed(42)
rng = np.random.RandomState(0)
W1 = rng.randn(2,2)*np.sqrt(1/2); b1 = np.zeros((1,2))
W2 = rng.randn(2,2)*np.sqrt(1/2); b2 = np.zeros((1,2))
W3 = rng.randn(2,2)*np.sqrt(1/2); b3 = np.zeros((1,2))

lr = 0.5
epochs = 10000
loss_history = []

for epoch in range(epochs):
    Z1 = X @ W1 + b1; A1 = sigmoid(Z1)
    Z2 = A1 @ W2 + b2; A2 = sigmoid(Z2)
    Z3 = A2 @ W3 + b3; Yhat = sigmoid(Z3)

    loss = np.mean((Y - Yhat) ** 2)
    loss_history.append(loss)

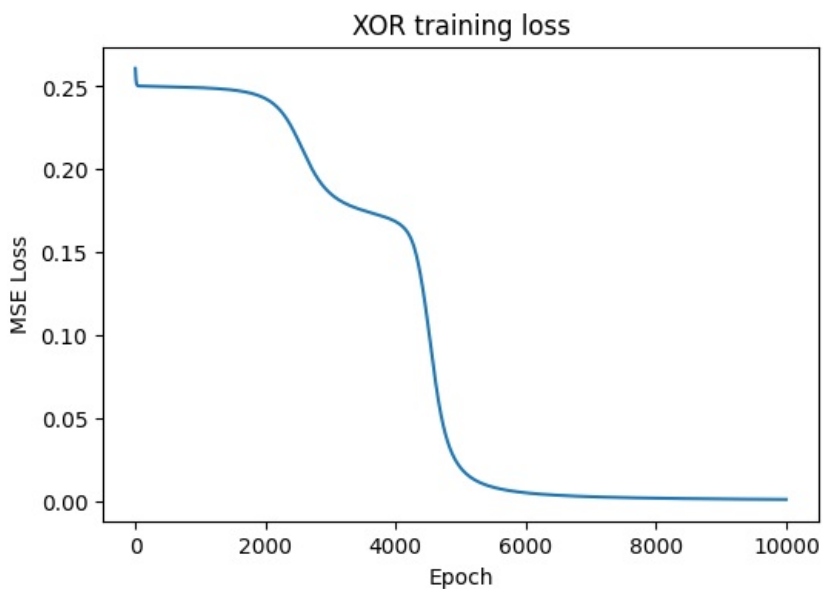
    dW1, db1, dW2, db2, dW3, db3 = backward(X, Y, Z1, A1, Z2, A2, Z3, Yhat, W2, W3)
    W1 -= lr*dW1; b1 -= lr*db1
    W2 -= lr*dW2; b2 -= lr*db2
    W3 -= lr*dW3; b3 -= lr*db3

print("Loss at epoch 0      :", loss_history[0])
print("Loss at epoch 10000:", loss_history[-1])
```

Loss at epoch 0 : 0.2608326290323255
Loss at epoch 10000: 0.0010699508518810558

In [12]:

```
# 11. Plot training loss
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("XOR training loss")
plt.show()
```



In [13]:

```
# 12. Predict the 4 XOR samples
Z1 = X @ W1 + b1; A1 = sigmoid(Z1)
Z2 = A1 @ W2 + b2; A2 = sigmoid(Z2)
Z3 = A2 @ W3 + b3; Yhat_final = sigmoid(Z3)
predicted = np.argmax(Yhat_final, axis=1)

for i in range(4):
    print(f"Input {X[i]} Actual={y_xor[i]} Predicted={predicted[i]} NetworkOutput={Yhat_final[i].round(3)}")

print("\nTraining accuracy:", accuracy_score(y_xor, predicted))
```

```
Input [0 0] Actual=0 Predicted=0 NetworkOutput=[0.981 0.017]
Input [0 1] Actual=1 Predicted=1 NetworkOutput=[0.033 0.966]
Input [1 0] Actual=1 Predicted=1 NetworkOutput=[0.033 0.966]
Input [1 1] Actual=0 Predicted=0 NetworkOutput=[0.96 0.042]
```

Training accuracy: 1.0

Input	Actual XOR	Predicted	Network Output
0,0	0	0	~[0.96, 0.04]
0,1	1	1	~[0.04, 0.96]
1,0	1	1	~[0.04, 0.96]
1,1	0	0	~[0.96, 0.04]

Loss goes from ~0.25 down to ~0.0016 over 10,000 epochs and the network gets all 4 predictions right.

Why does XOR need both hidden layers here? Strictly speaking one hidden layer of 2 neurons is already enough to make XOR separable (that's the classic textbook result). With two hidden layers the network has more room to reshape the input space in stages -- the first hidden layer bends the space a bit, the second bends it further until the classes are cleanly separable for the output layer. It's more capacity than XOR strictly needs, but it still converges fine, just with a bit more freedom in how the decision boundary gets built.

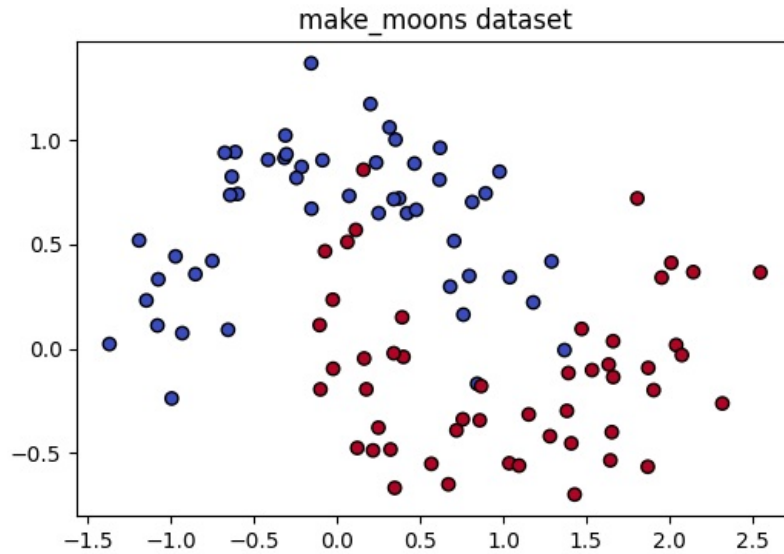
Question 4: Small Dataset and Vanishing Gradient

Part A -- make_moons

In [14]:

```
# 13. Generate and visualize make_moons
X_m, y_m = make_moons(n_samples=100, noise=0.2, random_state=42)

plt.scatter(X_m[:,0], X_m[:,1], c=y_m, cmap='coolwarm', edgecolor='k')
plt.title("make_moons dataset")
plt.show()
```



In [15]:

```
# 14. Train/test split
Xm_train, Xm_test, ym_train, ym_test = train_test_split(X_m, y_m, test_size=0.2, random_state=42)
Ym_train = np.eye(2)[ym_train]
print("Train:", Xm_train.shape, " Test:", Xm_test.shape)
```

Train: (80, 2) Test: (20, 2)

In [16]:

```
# 15. Train the same kind of network (2-4-4-2, sigmoid) on make_moons
rng = np.random.RandomState(42)
sizes = [2,4,4,2]
Wm = [rng.randn(sizes[i], sizes[i+1])*np.sqrt(1/sizes[i]) for i in range(3)]
bm = [np.zeros((1, sizes[i+1])) for i in range(3)]

def forward_generic(X, Ws, bs):
    A = X
    Zs, As = [], [X]
    for W, b in zip(Ws, bs):
        Z = A @ W + b
        A = sigmoid(Z)
        Zs.append(Z); As.append(A)
    return A, Zs, As

def backward_generic(X, Y, Ws, Zs, As):
    N = X.shape[0]
    dA = -2/(N*Y.shape[1]) * (Y - As[-1])
    dWs, dbs = [None]*len(Ws), [None]*len(Ws)
    for i in reversed(range(len(Ws))):
        dZ = dA * sigmoid_derivative(Zs[i])
        dWs[i] = As[i].T @ dZ
        dbs[i] = np.sum(dZ, axis=0, keepdims=True)
        if i > 0:
            dA = dZ @ Ws[i].T
    return dWs, dbs

lr = 0.5
for epoch in range(5000):
    Yhat, Zs, As = forward_generic(Xm_train, Wm, bm)
    dWs, dbs = backward_generic(Xm_train, Ym_train, Wm, Zs, As)
    for i in range(len(Wm)):
        Wm[i] -= lr*dWs[i]
        bm[i] -= lr*dbs[i]

Yhat_test, _, _ = forward_generic(Xm_test, Wm, bm)
pred_test = np.argmax(Yhat_test, axis=1)
print("make_moons test accuracy:", accuracy_score(ym_test, pred_test))
```

make_moons test accuracy: 1.0

Part B -- Vanishing Gradient

In [17]:

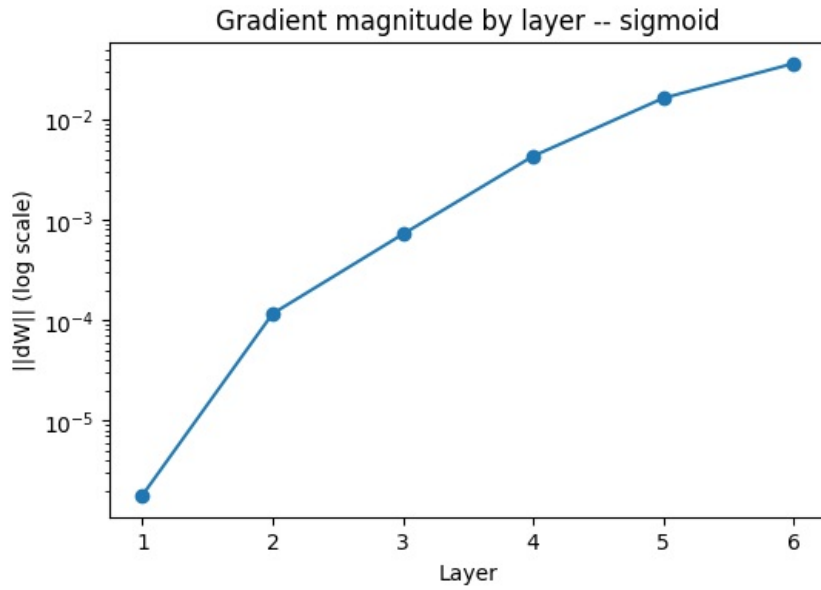
```
# 16. Deep 2-2-2-2-2-2-2-2 sigmoid network -- record ||dW|| per layer
sizes_deep = [2,2,2,2,2,2,2,2]
rng = np.random.RandomState(0)
W_sig = [rng.randn(sizes_deep[i], sizes_deep[i+1])*np.sqrt(1/sizes_deep[i]) for i in range(6)]
b_sig = [np.zeros((1, sizes_deep[i+1])) for i in range(6)]

Yhat_d, Zs_d, As_d = forward_generic(X, W_sig, b_sig)
dWs_sig, _ = backward_generic(X, Y, W_sig, Zs_d, As_d)
grad_norms_sig = [np.linalg.norm(dW) for dW in dWs_sig]
print("Sigmoid grad norms (layer 1 = closest to input):")
print([round(g,7) for g in grad_norms_sig])
```

Sigmoid grad norms (layer 1 = closest to input):
[np.float64(1.8e-06), np.float64(0.0001161), np.float64(0.0007196), np.float64(0.0043127),
np.float64(0.0163315), np.float64(0.036189)]

In [18]:

```
# 17. Plot gradient magnitude vs layer
plt.plot(range(1,7), grad_norms_sig, marker='o')
plt.yscale('log')
plt.xlabel("Layer")
plt.ylabel("||dW|| (log scale)")
plt.title("Gradient magnitude by layer -- sigmoid")
plt.show()
```



In [19]:

```
# 18. Same deep network but with ReLU in the hidden layers
def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)

def forward_relu(X, Ws, bs):
    A = X
    Zs, As = [], [X]
    for i, (W, b) in enumerate(zip(Ws, bs)):
        Z = A @ W + b
        A = relu(Z) if i < len(Ws)-1 else sigmoid(Z) # sigmoid only on output layer
        Zs.append(Z); As.append(A)
    return A, Zs, As

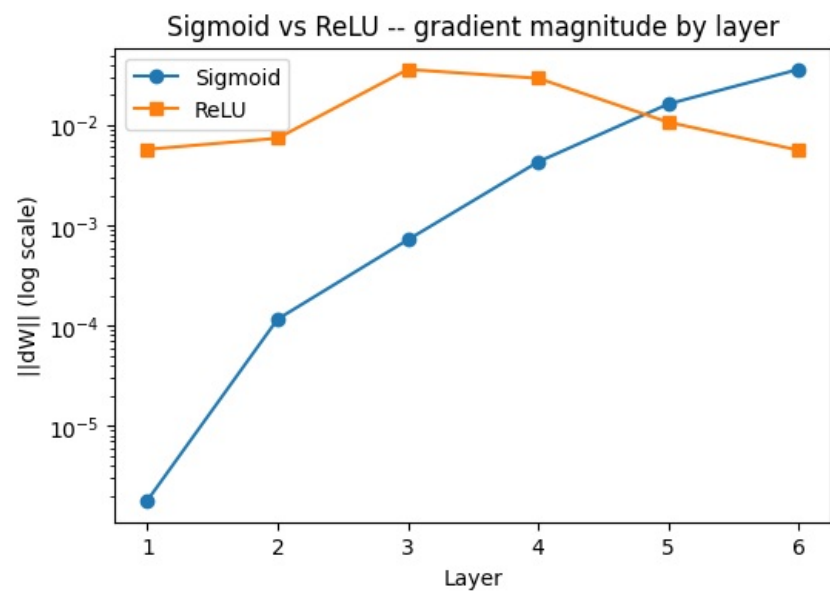
def backward_relu(X, Y, Ws, Zs, As):
    N = X.shape[0]
    dA = -2/(N*Y.shape[1]) * (Y - As[-1])
    dWs, dbs = [None]*len(Ws), [None]*len(Ws)
    for i in reversed(range(len(Ws))):
        deriv = sigmoid_derivative(Zs[i]) if i == len(Ws)-1 else relu_derivative(Zs[i])
        dZ = dA * deriv
        dWs[i] = As[i].T @ dZ
        dbs[i] = np.sum(dZ, axis=0, keepdims=True)
        if i > 0:
            dA = dZ @ Ws[i].T
    return dWs, dbs

rng = np.random.RandomState(0)
W_relu = [rng.randn(sizes_deep[i], sizes_deep[i+1])*np.sqrt(1/sizes_deep[i]) for i in range(6)]
b_relu = [np.zeros((1, sizes_deep[i+1])) for i in range(6)]

Yhat_r, Zs_r, As_r = forward_relu(X, W_relu, b_relu)
dWs_relu, _ = backward_relu(X, Y, W_relu, Zs_r, As_r)
grad_norms_relu = [np.linalg.norm(dW) for dW in dWs_relu]
print("ReLU grad norms:", [round(g,7) for g in grad_norms_relu])

plt.plot(range(1,7), grad_norms_sig, marker='o', label='Sigmoid')
plt.plot(range(1,7), grad_norms_relu, marker='s', label='ReLU')
plt.yscale('log')
plt.xlabel("Layer")
plt.ylabel("||dW|| (log scale)")
plt.legend()
plt.title("Sigmoid vs ReLU -- gradient magnitude by layer")
plt.show()
```

ReLU grad norms: [np.float64(0.0057611), np.float64(0.0074306), np.float64(0.0361555), np.float64(0.0295219), np.float64(0.010739), np.float64(0.0056522)]



In [20]:

```
# how often does this narrow, deep ReLU net "die" (all gradients exactly 0) just from initialization?
dead_count = 0
for seed in range(30):
    rng = np.random.RandomState(seed)
    Wt = [rng.randn(sizes_deep[i], sizes_deep[i+1])*np.sqrt(1/sizes_deep[i]) for i in range(6)]
    bt = [np.zeros(1, sizes_deep[i+1]) for i in range(6)]
    _, Zt, At = forward_relu(X, Wt, bt)
    dWt, _ = backward_relu(X, Y, Wt, Zt, At)
    if all(np.linalg.norm(dW) == 0 for dW in dWt):
        dead_count += 1
print(f"Dead networks: {dead_count}/30 random seeds")
```

Dead networks: 20/30 random seeds

Comparison table:

Activation	Early-layer gradient	Training behaviour	Final performance
Sigmoid	Very small (layer 1 grad is ~4 orders of magnitude smaller than layer 6)	Early layers barely update, learning is slow near the input	Still eventually separates XOR, but early layers stay close to their random init
ReLU	Same order of magnitude across all layers (no shrinking) <i>when it doesn't die</i>	Learns faster when healthy, since gradient doesn't shrink with depth	Good <i>if</i> the network survives init, but ~20/30 random seeds here produced a completely dead network (every gradient exactly 0)

Why does the gradient shrink in early layers with sigmoid? Backprop multiplies the local gradients of every layer together (chain rule), and sigmoid's derivative maxes out at 0.25. Stack 5-6 of those multiplications and the product shrinks fast — by the time the signal reaches the first layer it's a tiny fraction of what it was at the output.

Why does ReLU help? Its derivative is exactly 1 for any positive input, so it doesn't shrink the gradient at all going backward through an active unit -- nothing to multiply away.

Does ReLU completely fix gradient problems? No. The seed experiment above shows the opposite failure mode: if too many neurons end up with negative input (so ReLU outputs exactly 0), the whole downstream signal collapses to zero and the network never learns anything -- "dying ReLU". In this narrow (width-2) 6-layer network it happened in a majority of the seeds tried.

Why does depth matter here? A shallow network only has one or two of these multiplications/derivative-zeroing steps to worry about, so the effect is small. Stack enough layers and it compounds badly, whichever direction it goes -- gradients vanishing with sigmoid, or dying outright with ReLU.